

# Corner and Centroid Detection

## Corner and Centroid Detection

Preface

Example Results

Principle Explanation

What are Image Moments and Centroids

Why Polygon Approximation Can Find Corners

Code Overview

Initialize Camera and Display Module

Set Contour Area Threshold

Grayscale, Binarization, and Contour Extraction

Calculate and Draw Centroids

Extract and Draw Corners

Display, Statistics, and Resource Release

Parameter Adjustment Notes

## Preface

This tutorial is applicable to firmware version: CanMV\_K230\_YAHBOOM\_micropython\_v1.8-13. Download from Jianan official website [https://download.kendryte.com/developer/releases/canmv\\_k230\\_micropython/daily\\_build/](https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/)

 **K230 MicroPython 镜像下载**

CI/CD 更新时间: 2026-07-28 01:29:59

选择开发板

 CanMV K230 V1.0/V1.1

 CanMV K230 V3.0

 庐山派 K230

 庐山派 Lite K230D

 **Yahboom K230**

 01Studio K230

 01Studio K230 Emmc

 Hiwonder K230

 WonderMK K230

 GT6700 K230

 **勘智 KENDRYTE**

**Yahboom K230**  
MicroPython Build for Yahboom K230  
当前下载镜像:  
[CanMV\\_K230\\_YAHBOOM\\_micropython\\_v1.8-28-g9b151af\\_nncase\\_v2.11.0.img.gz](#)

最新每日构建版本  
**Daily-0728**  
编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa  
[立即下载](#)

**极简 USB 烧录指南 (推荐)**

⚠ 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具  
下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)  

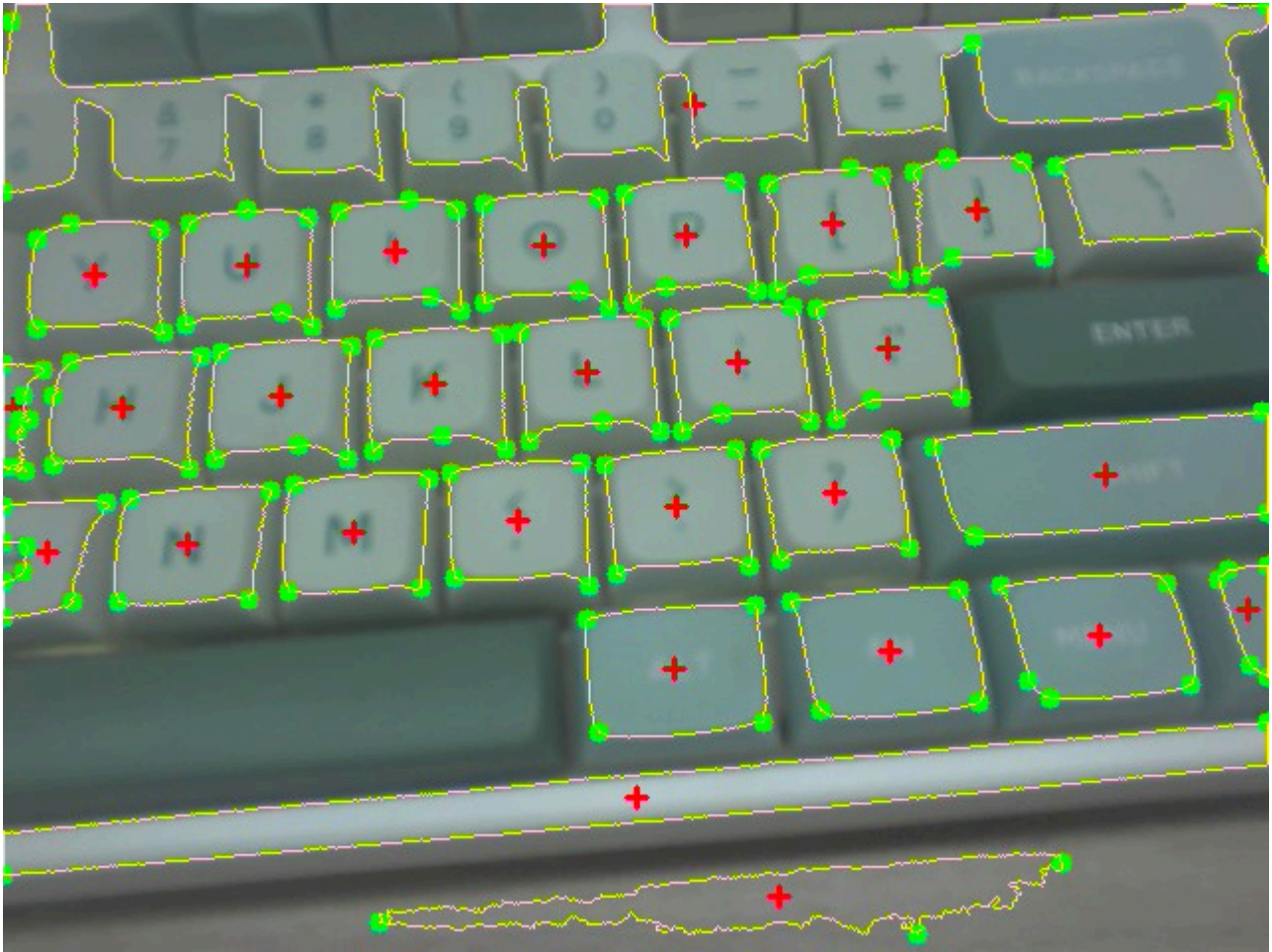
☒ 有 BOOT 键的板子

☐ 无 BOOT 键的板子

## Example Results

Run the example code for this section [04.camera\\_corners.py](#).

This section performs grayscale conversion and OTSU binarization on the real-time camera feed, extracts external contours, calculates contour centroids using image moments, and obtains contour corners using polygon approximation. The centroid is marked with a blue cross, and corners are marked with green solid circles.



## Principle Explanation

### What are Image Moments and Centroids

Image moments are a set of features obtained by weighted summation of contour coordinates and their distributions. The zeroth-order moment  $m_{00}$  corresponds to the contour area, and the first-order moments  $m_{10}$  and  $m_{01}$  can be used to calculate the centroid:

$$\begin{aligned} cx &= m_{10} / m_{00} \\ cy &= m_{01} / m_{00} \end{aligned}$$

When  $m_{00}$  is 0, division is not possible, so the code must first check if  $m_{00} > 0$ .

### Why Polygon Approximation Can Find Corners

`approxPolyDP()` uses the Douglas-Peucker algorithm to simplify contours, retaining only the points that have significant impact on the overall shape. A smaller error threshold preserves more points; a larger threshold simplifies the contour more. In this example, the error is set to 2% of the contour perimeter.

Note that this example obtains "contour approximation vertices", not texture corners in the sense of `goodFeaturesToTrack()` or Harris algorithm.

## Code Overview

---

### Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

### Set Contour Area Threshold

```
MIN_A = 200
```

Contours with area less than 200 pixels are ignored directly to reduce corners from binary noise.

### Grayscale, Binarization, and Contour Extraction

```
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)
contours, _ = cv2.findContours(
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

OTSU auto-threshold is suitable for scenes where foreground and background have significantly different grayscale distributions. `RETR_EXTERNAL` only takes external contours, and internal holes will not be processed as independent contours.

### Calculate and Draw Centroids

```
moments = cv2.moments(contour)
if moments["m00"] > 0:
    cx = int(moments["m10"] / moments["m00"])
    cy = int(moments["m01"] / moments["m00"])
    cv2.drawMarker(
        img_np, (cx, cy),
        (255, 0, 0),
        cv2.MARKER_CROSS, 10, 2
    )
```

Centroid coordinates are converted to integers for drawing. The color tuple is passed according to the current image channel order.

## Extract and Draw Corners

```
perimeter = cv2.arcLength(contour, True)
approx = cv2.approxPolyDP(contour, 0.02 * perimeter, True)

for point in approx:
    x = int(point[0][0])
    y = int(point[0][1])
    cv2.circle(img_np, (x, y), 5, (0, 255, 0), -1)

cv2.drawContours(img_np, [contour], -1, (255, 255, 0), 1)
```

`arcLength(contour, True)` treats the contour as a closed curve. The source code processes up to 50 valid contours at most to prevent excessive drawing from complex backgrounds.

## Display, Statistics, and Resource Release

```
Display.show_image(img)

if time.ticks_diff(time.ticks_ms(), ft) >= 1000:
    print("corners:", corner_count, "| fps:", fc)

finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

## Parameter Adjustment Notes

- **MIN\_A:** Decrease when small target corners are filtered out, increase when there are too many small contours in the background.
- **OTSU Foreground Polarity:** If the target is divided into black, background external contours may be detected. Use `THRESH_BINARY_INV` or a fixed threshold.
- **Approximation Error 0.02:** Increase when there are too many corners, decrease when corners are being merged.
- **Contour Upper Limit:** The source code limit is 50. Actual projects should use ROI and area range to reduce candidate contours in advance.
- **Texture Corners:** When detecting chessboard patterns, textures, or corners inside objects, use `goodFeaturesToTrack()` instead of treating contour vertices as corners for all types.